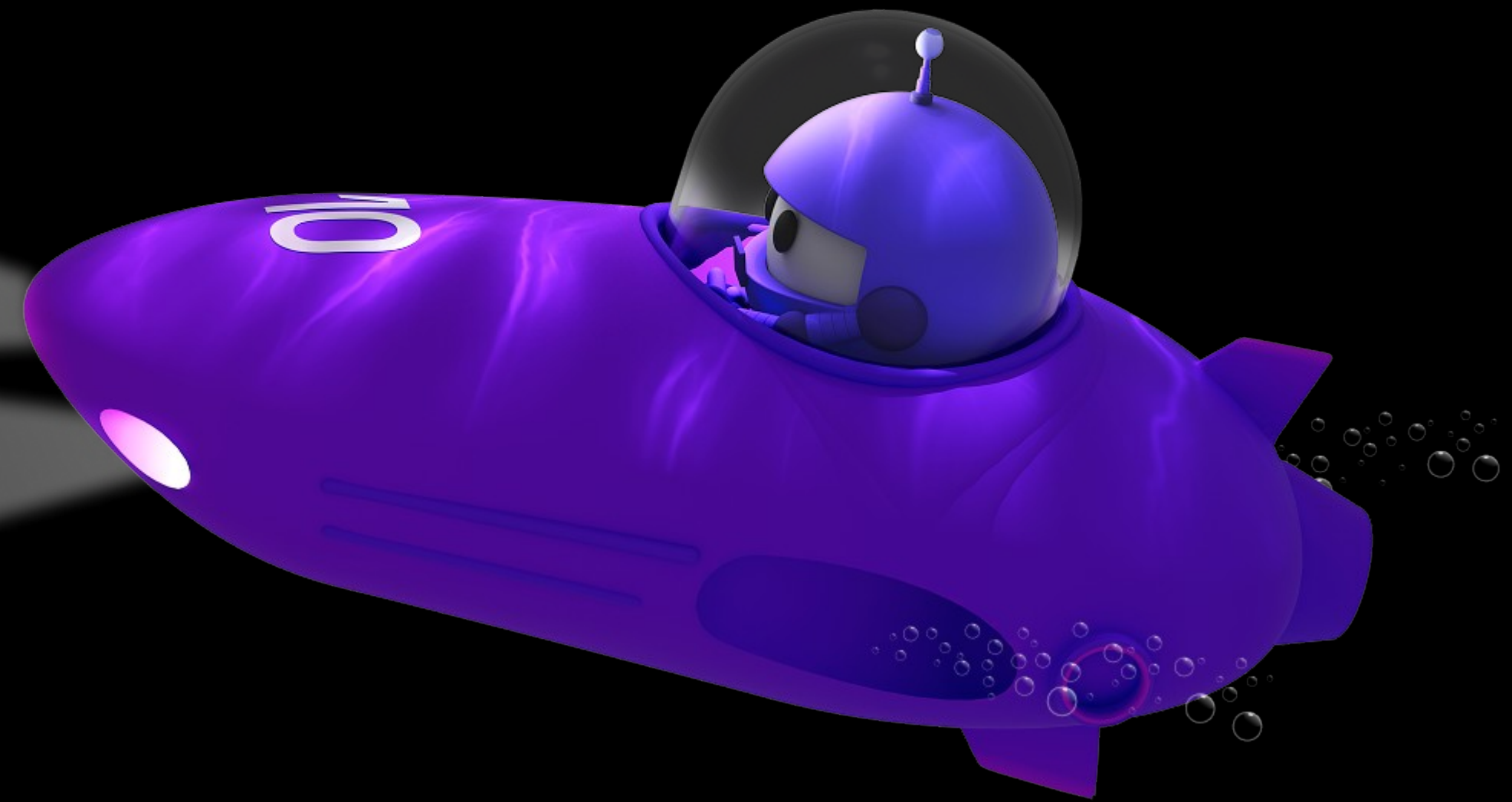


From Zero to Hero

C# 파일 하나로 시작하는
AI 친화적인 초고속 .NET 앱
개발

.NET





남정현
.NET Dev

이제 C# 도
파이썬 감성으로 ,
코딩하세요 .

시작하며

파일 기반 앱이란 ?

- ✓ Visual Studio 같은 IDE 없이
 - ✓ VS Code 또는 여러분이 선호하는 OSS 에디터, 메모장 하나로
 - ✓ C# 코드를 작성하고 `dotnet run` 명령어로 파일을 실행하기만 하면
- 프로그램이 짤 하고 실행되는 방식입니다 .

시작하며

개발 환경 셋팅은 어떻게 해요 ?

메모장을 사랑하시나요 ?

Ⓟ 인텔리센스는 없어도 코드 복붙 / 암기하실 수 있으면 메모장으로도 충분합니다 !

VS Code 를 사용하시나요 ?

➔ C# Dev Kit 을 설치하고 프리뷰 기능을 켭니다 .

다른 에디터 * 를 쓰시나요 ?

➔ dotnetdev-kr-custom/csharp 플러그인을 설치하세요 .

* OpenVSX 피드를 쓰는 모든 Code OSS 기반 에디터에서 사용 가능 -

VSCodium, Cursor, Google Antigravity, Windsurf, Eclipse Theia, Bytedance Trae, code-server, Google Firebase Studio, Void, Amazon Kiro 등...

Not
Required



Ⓟ

With

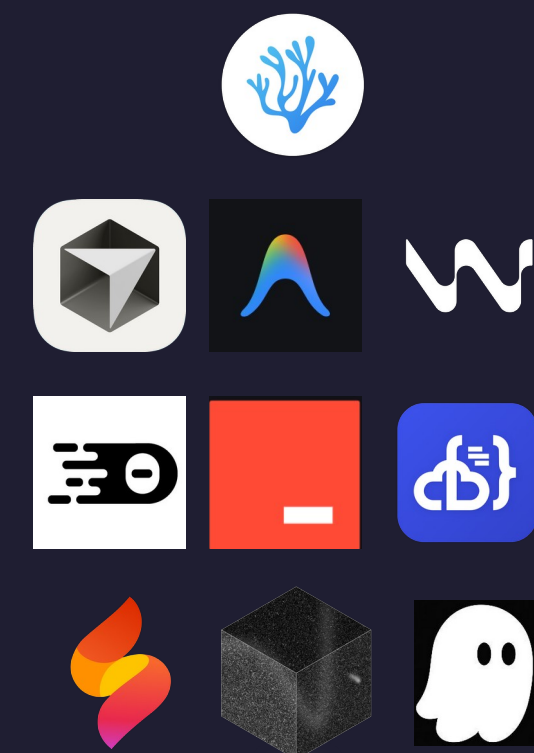


+

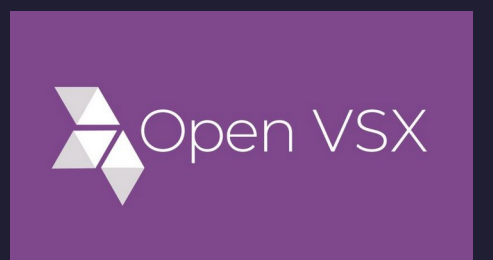


Or

With



+



시작하며

지금 노트북 가지고 계시죠 ?

듣기만 하면 지겨우니까 , 잠깐 닷넷 설치하고 가실게요 .

- 맥 쓰시는 분 :
\$ brew install --cask dotnet-sdk # Homebrew 쓰시는 분
\$ sudo port install dotnet-sdk-10 # MacPort 쓰시는 분
- 우분투 쓰시는 분 : (나중에는 backport 피드 구독 없이도 그냥 설치 되실겁니다 .)
\$ sudo add-apt-repository ppa:dotnet/backports
\$ sudo apt update && sudo apt install -y dotnet-sdk-10.0
- 윈도우 쓰시는 분 :
> winget install Microsoft.DotNet.SDK.10

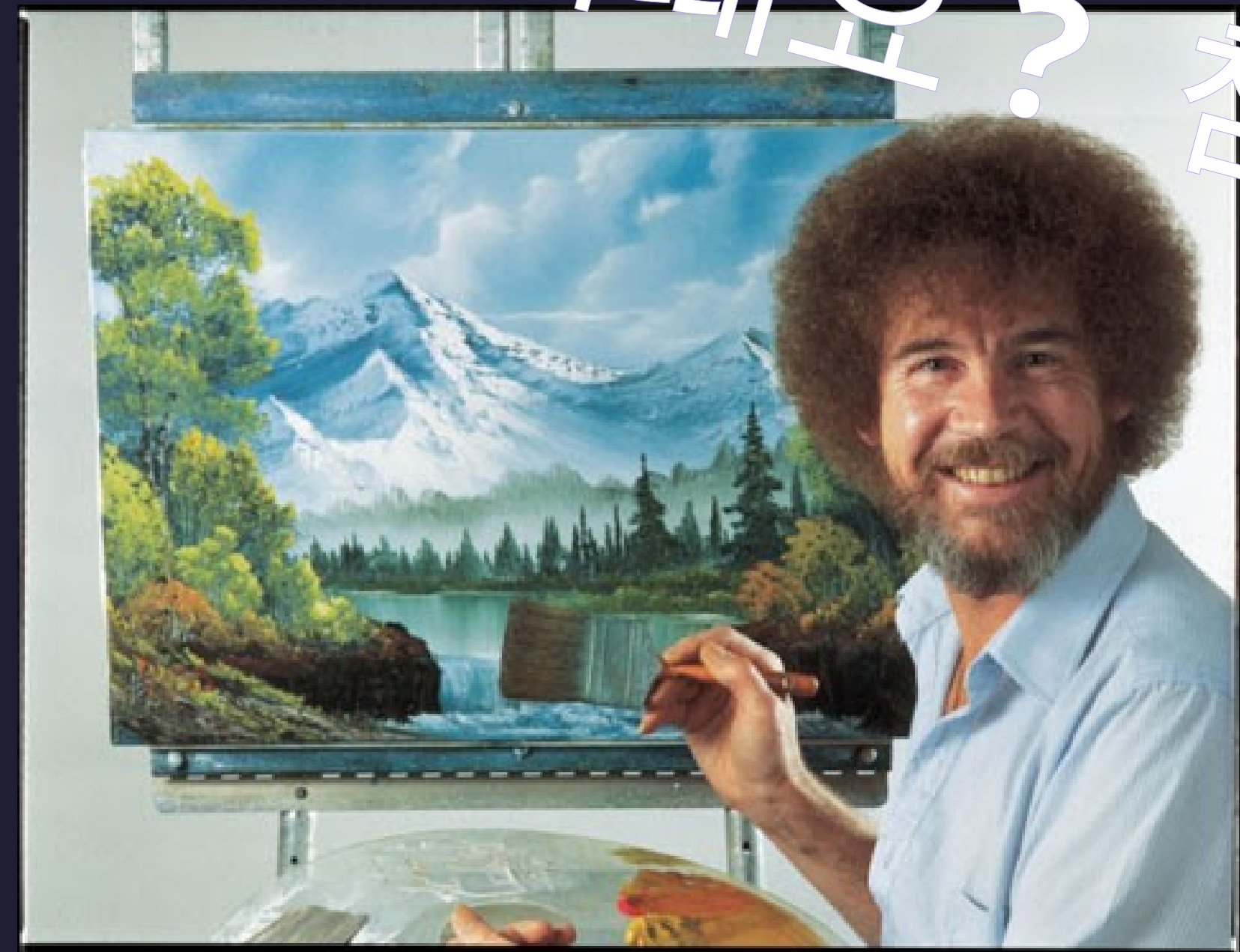
설치 다 하시고 , dotnet --list-sdks 실행해서 10.0 이 나오면 설치가 잘 되신 겁니다 .

시작하며

파일 기반 앱으로 코딩하기 (5 단계)

1. 닷넷 SDK 설치하신 후에 에디터 (혹은 메모장) 를 열고 ,
2. `Console.WriteLine("Hello, World!");` 를 입력하고
3. 파일을 `Hello.cs` 로 저장한 다음
4. `dotnet run Hello.cs` 라고 입력하면
5. `Hello, World!` 라고 뜹니다 .

어때요? 참 쉽죠



시작하며

같은 일을 Visual Studio
계?)

1. Visual Studio 를 열
2. 새로 만들기 @ 프로
3. 검색 란에 “콘솔 프
4. 언어 종류는 “C#”
5. 콘솔 앱이 나오긴 하
(.NET Framework
6. 프로젝트 이름을 H
누릅니다.
7. 프레임워크는 .NET
8. ...



하셔야 겠죠 ?)

선택합니다 .

합니다 .

이다 .

하고 , 다음 버튼을

다음 버튼을 누릅니다 .

시작하며

파일 기반 앱이란 ?

C# 은 여전히 프로젝트와 솔루션 기반의 빌드 환경을 사용합니다 .
그러나 , 파일 기반 앱은 ...

파일 1 개에서 CSPROJ 의 기능과 C# 코드를 모두 통합했습니다 .
dotnet run 명령어만 넣어주면 .NET CLI 가 알아서 컴파일 /
실행해줍니다 .

무엇보다도 파일 기반 앱은 리눅스 , 맥에서 더 잘 돌아갑니다 .

C#/.NET 좀 하셨다고요?

**신 문물은 이게 끝이
아닙니다.**

큰 거 들어갑니다!

셸 스크립트 만들기

셸 스크립트 만들기

닷넷으로 만드는 셸 스크립트

파일 기반 앱 한정으로 새로운 문법이 추가되었습니다 .

```
#!/usr/bin/env dotnet  
Console.WriteLine("No lemon, no melon.");
```

이렇게 내용을 만든 후 `palindrome` (확장자 없이) 이라고 저장합니다 . 그리고...

```
$ chmod +x ./palindrome  
$ ./palindrome
```

이렇게 실행해봅시다 .

아시죠 ? 확장자 없어도 Linux
와 맥에서는 셸 스크립트 실행
됩니다 . 😊

셸 스크립트 만들기

닷넷으로 만드는 셸 스크립트

원래 C# 문법에서 ‘#’으로 시작하는 기호는 매크로 / 전처리기에서만 제한적으로 지원되던 문법이었습니다.

그러나 파일 기반 앱의 강력한 기능을 위해, 원래 문법에 없던 내용을 문법적인 유연함을 발휘하여 마치 셸 스크립트처럼 쓸 수 있게 만들어주었습니다.

만약 ‘#’ 대신 ‘//’로 시작하는 Shebang Line 을 원하신다면, 아래처럼 쓸 수도 있습니다.

어디서 저렇게 쓰냐 하면, Jbang 이 이런 식으로 Shebang 라인을 씁니다.

```
#!/usr/bin/env dotnet "$0" "$@" ; exit $?  
Console.WriteLine("No lemon, No melon.");
```

셸 스크립트 만들기

닷넷으로 만드는 셸 스크립트

그럼 Shebang 라인을 어떻게 쓰는 이유, 또한 어떻게 쓸 수 있는 원리는 무엇일까요?

- dotnet CLI 는 10 에 들어와서 파일 기반 앱을 위한 기능을 새로 추가했습니다.
- 바로 표준 입력으로 들어오는 문자열을 파일 기반 앱으로 처리하는 기능입니다.
- 그리고 `/usr/bin/env` 는 현재 셸 스크립트를 실행하는 유저 기준으로 경로 해석을 돕습니다.
- .NET SDK 와 CLI 가 제대로 설치되어 있기만 하면 거의 확실하게 실행되는 것이죠!

그러면 셸 스크립트의 경로도 얻어올 수 있나요? 네, 가능합니다!

- `var filePath = AppContext.GetData("EntryPointFilePath") as string;`
- `var directoryPath = AppContext.GetData("EntryPointFileDirectoryPath") as string;`
- 위의 코드는 파일 기반 앱 한정으로 유효한 값을 반환하는 코드입니다.

Demo: 셸 스크립트 실행

셸 스크립트 만들기

스크립트 더 쉽게 실행하기

Homebrew 설치할 때는 흔히 아래처럼 실행하실 겁니다 .

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.s  
h)"
```

비슷한 일을 이제 닷넷에서도 얼마든지 하실 수 있습니다 .

Shebang 라인이 없어도 되는 기능입니다 .

```
curl -s -L https://is.gd/0zV2lz -o - | dotnet run -
```

(Windows 에서는 curl 대신 curl.exe 로 써주시면 됩니다 ! 2018 년 가을부터
내장되었습니다 .)

Fast API 감성으로 웹 API 만들어보기

닷넷의 웹 기술

여러분이 아는 닷넷의 웹 기술은 다음 중 어떤 것인가요?

- ASP: 닷넷 시대는 아니지만, IIS 위에서 실행하는 옛날 웹 기술
- Web Form (+ AJAX): VB 6, 델파이 같은 RAD 애플리케이션 개발 기법을 웹 에서도 쓸 수 있게 도와주는 프레임워크.
- MVC: Ruby on Rails 에서 영감을 받아 Model, View, Controller 3 개 요소 간의 협업으로 빠르게 웹 애플리케이션 개발을 할 수 있게 도와주는 프레임워크.
- Web Page: 컨트롤러 개념은 따로 두지 않고, 페이지와 페이지 모델로 구성된 웹 페이지 중심의 개발 프레임워크.

오늘 이야기하려는 것은 위의 모든 기술들을 뛰어넘은 최첨단 웹 기술입니다.
바로 Minimal API 입니다.

Fast API 감성으로 웹 API 만들기

일단 시작에 앞서, AOT

• **이옛날 어른들은**

한 서버에서 수십, 수백개의 모놀리식 애플리케이션을 돌렸습니다.

• **요즈음 어른들은**

서버리스, 마이크로서비스, 경량 API 수 천개를 수 백대의 서버에서 수 만개의 컨테이너위에 나누어 돌립니다.

이는 곧,

- 컨테이너 /Pod 하나가 하는 일이 작고 짧고 빠르며,
- 사용자 요청이 생길 때만 지연 없이 빠르게 실행되며,
- 유지 비용을 최소화해야 함을 의미합니다.

Fast API 감성으로 웹 API 만들기

일단 시작에 앞서, AOT
이야기부터.

AOT를 쓰면,

- 처음부터 모든 코드가 기계어 코드로 컴파일되므로 JIT가 개입하지 않습니다.
- 프로그램 코드가 변화할 여지가 없으므로 기계적으로 필요한 코드만 남길 수 있습니다.

따라서 AOT는 “클라우드·컨테이너·서버리스 시대에 빠르고 작고 저렴한 애플리케이션을 만들기 위한 핵심 지향점”이 됩니다.

닷넷의 웹 기술

왜 Minimal API 를 써야 할까요 ?

- **FastAPI 처럼 명확한 엔드포인트 선언** ☾
각 라우트를 함수 / 람다로 직접 매핑하며 , DI(의존성 주입) 도 그대로 적용됩니다 .
- **MVC 처럼 리플렉션 · 런타임 매직에 의존하지 않음** ☾
Minimal API 는 정적으로 선언된 코드만 사용하므로 AOT/ 트리밍 / 분석에 결정적으로 유리합니다 .
- **결과적으로 작은 코드 , 예측 가능한 구조 , 유지보수 비용 절감** ☾
“보이지 않는 프레임워크 규칙”이 아니라 자신이 작성한 코드가 곧 계약이 됩니다 .

☞ FastAPI 가 인기 있는 이유

- 복잡한 프레임워크 규칙 없이 , 딱 보이는 만큼만 동작하기 때문입니다 .
Minimal API 도 같은 철학을 .NET 에 제공합니다 .

Fast API 감성으로 웹 API 만들기

코드 살펴보기

바로 코드를 살펴보겠습니다.

```
#!/usr/bin/env dotnet
#:sdk Microsoft.NET.Sdk.Web
#:property PublishAot=false
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<Random>(Random.Shared);
using var app = builder.Build();
app.MapGet("/", ([FromServices] Random rnd) =>
    new { ts = DateTimeOffset.UtcNow, val = rnd.Next() % 10, });
app.Run();
```

단 8 줄이면 난수 생성하는 웹 API 가 순식간에 만들어집니다.

여기엔 컨트롤러도, Startup 도, 클래스 파일 분리도 없습니다.

Demo: API 서버 띄우기

파일 기반 마이크로 서비스 (feat. Aspire, Podman)

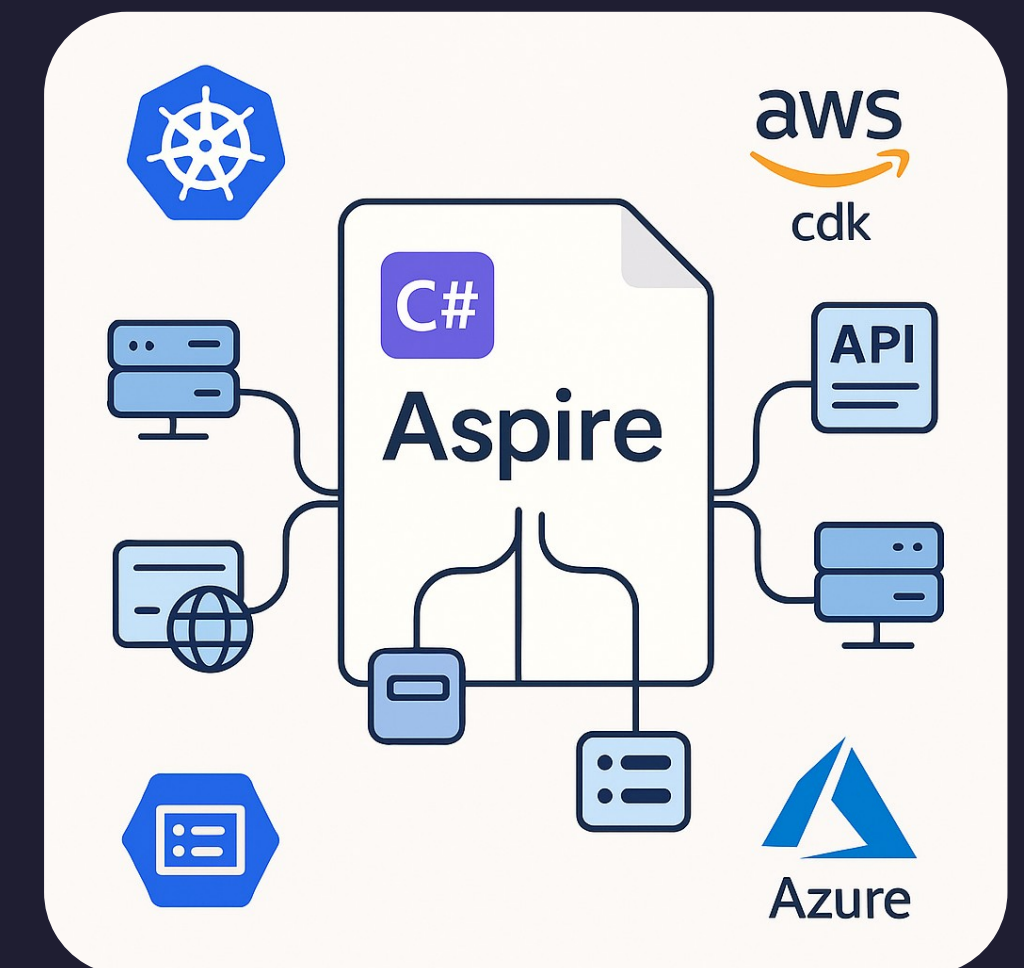
파일 기반 마이크로 서비스 (feat. Aspire, Podman)

파일 기반 마이크로서비스 ?

Aspire 는 개발 환경에서 오케스트레이션 · 배포 · 관측까지의 운영 작업을 C# 과 .NET 런타임 안에서 구성할 수 있게 돕는 프레임워크입니다 .

개발자는 오로지 코드에 집중하면 되고 , Aspire 는 Docker/Podman 기반 로컬 오케스트레이션 , 디버거 연결 , 대시보드 제공부터 Docker Compose, Kubernetes, AWS CDK, Azure 배포까지 **First/Last Mile** 전 과정을 자동으로 처리합니다 .

즉 , 개발자는 별도로 IaC 도구나 클라우드 도메인 언어를 깊이 학습하지 않더라도 , Aspire 가 개발 단계의 **inner loop** 에서 필요한 리소스 구성과 실행 환경을 대신 처리해 주어 애플리케이션 개발에 집중할 수 있게 돕습니다 .



파일 기반 마이크로 서비스 (feat. Aspire, Podman)

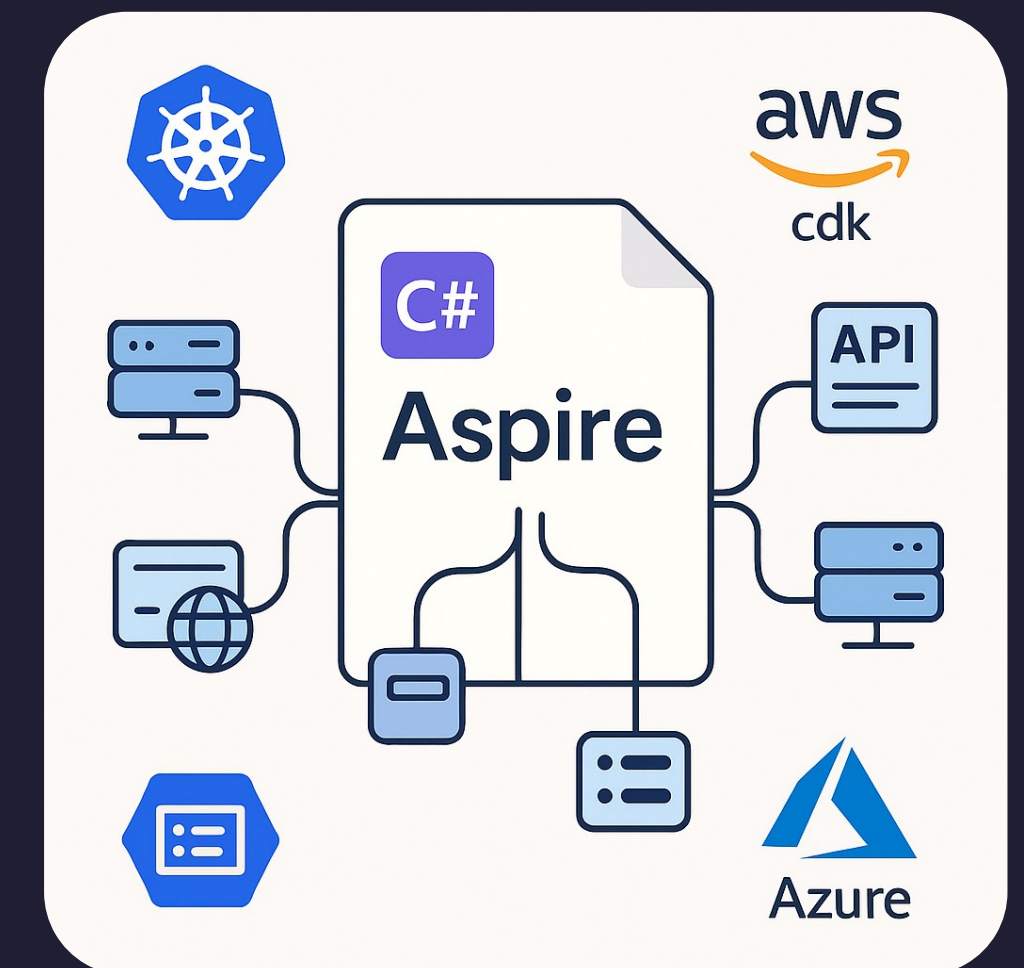
Podman? Docker 가 아니고 ?

Docker 는 컨테이너 생태계의 대표 기술이지만 , Windows 나 macOS 에서 사용하는 Docker Desktop 은 상용 제품이며 , 조건에 따라 별도 라이선스 구매가 필요합니다 .

반면 Podman Desktop 은 동일한 개발 환경을 제공하면서도 무료로 사용 가능하고 , Windows·macOS 에서 대부분의 시나리오에서 완전한 대안이 됩니다 .

특수한 Windows 네이티브 컨테이너를 다루는 경우가 아니라면 , 개발 환경에서는 **Podman** 을 권장합니다 .

Aspire, Visual Studio 2026 은 Docker 는 물론 Podman 도 아주 잘 지원하므로 추천합니다 .



파일 기반 마이크로 서비스 (feat. Aspire, Podman)

Aspire 13 에서 달라진 점

.NET Aspire 라는 이름에 익숙하신 분도 계실 텐데요 , .NET 10 출시와 함께 명칭이 **Aspire** 로 바뀌고 버전도 **13** 으로 올라갔습니다 .

이번 개편은 .NET 중심 구조는 유지하면서도 , **Python·Node** 등 다양한 애플리케이션을 유연하게 포함할 수 있는 오케스트레이션 플랫폼으로 확장하려는 방향성을 담고 있습니다 .

특히 Aspire 13 에서는 파일 기반 앱과 **C#** 부트스트래핑 모델이 더욱 강화되어 , 개발자가 별도의 프로젝트 템플릿이나 복잡한 구성 없이 코드만으로 애플리케이션과 인프라를 빠르게 오케스트레이션할 수 있게 되었습니다 .

파일 기반 마이크로 서비스 (feat. Aspire, Podman)

Aspire 샘플 (AppHost.cs)

```
#!/usr/bin/env dotnet
#:sdk Aspire.AppHost.Sdk@13.0.0
#:property PublishAot=false
#:package Aspire.Hosting.Garnet@13.0.0

#pragma warning disable ASPIRECSHARPAPPS001

using Microsoft.Extensions.Configuration;

// To specify password/secrets:
// dotnet user-secrets set --id=apphosttest
// ConfigName Value

var builder =
    DistributedApplication.CreateBuilder(args);
builder.Configuration
    .AddJsonFile("apphost.json")
    .AddUserSecrets("apphosttest")
    .AddEnvironmentVariables();
```

AppHost 는 보통의 .NET 프로그램과 다릅니다 . 별도의 SDK 를 사용해야 합니다 .

시작 부분만 다르게 지정하고 , 나머지는 일반적인 파일 기반 앱 처럼 실행할 수 있게 만들어주면 대시보드 , 부트스트래핑 , 오케스트레이션이 자동으로 이루어집니다 .

파일 기반 마이크로 서비스 (feat. Aspire, Podman)

Aspire 샘플 (AppHost.cs)

```
var garnet = builder.AddGarnet("cache");

_ = builder.AddCSharpApp("worker", "worker.cs")
    .WithReference(garnet)
    .WaitFor(garnet);

_ = builder.AddCSharpApp("minapi", "minapi.cs")
    .WithReference(garnet)
    .WaitFor(garnet)
    .WithHttpsEndpoint()
    .WithHttpEndpoint();

using var app = builder.Build();
app.Run();
```

Microsoft 의 Redis 호환
구현체인 Garnet 은 C#/.NET
으로 만들어져있고 , Aspire
위에도 아주 잘 올라갑니다 .

이 컴포넌트를 두고
Worker.cs , Minapi.cs 가
서로 통신하는데 Garnet 을
사용합니다 .

파일 기반 마이크로 서비스 (feat. Aspire, Podman)

Aspire 샘플 (AppHost.json)

```
{
  "DOTNET_ENVIRONMENT": "Development",
  "ASPNETCORE_URLS": "http://localhost:18888",
  "ASPIRE_DASHBOARD_OTLP_ENDPOINT_URL":
"http://localhost:18889",
  "ASPIRE_DASHBOARD_OTLP_HTTP_ENDPOINT_URL":
"http://localhost:18890",
  "ASPIRE_ALLOW_UNSECURED_TRANSPORT": "true",

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime":
"Information"
    }
  }
}
```

Aspire AppHost 도 일반적인
닷넷 애플리케이션과
마찬가지로 환경 변수, .NET
User Secret, JSON 설정파일
등을 통한 설정 주입이
가능합니다.

Aspire Dashboard 설정까지는
포함되어야 Aspire 구동이
가능합니다.

파일 기반 마이크로 서비스 (feat. Aspire, Podman)

Aspire 샘플 (MinApi.cs)

```
#!/usr/bin/env dotnet
#:sdk Microsoft.NET.Sdk.Web
#:property PublishAot=false
#:package Aspire.StackExchange.Redis@13.0.0

using Microsoft.AspNetCore.Mvc;
using StackExchange.Redis;

var builder =
WebApplication.CreateBuilder(args);
builder.AddRedisClient("cache");
using var app = builder.Build();
```

Aspire 에서 중요한 기능 중 하나는 서로 다른 컴포넌트가 어떻게 서로를 찾고 통신하는가 입니다 .

AppHost.cs 에서 정의한 리소스 이름과 엔드포인트 이름은 , 각 애플리케이션 코드에서 그대로 상수로 사용해도 되며 , 개발 환경부터 클라우드 배포까지 동일하게 유효합니다 .

이러한 방식은 .NET 뿐만 아니라 **Python, Node** 를 포함해 **Aspire** 가 오케스트레이션하는 모든 애플리케이션에 동일하게 적용됩니다 .

파일 기반 마이크로 서비스 (feat. Aspire, Podman)

Aspire 샘플 (MinApi.cs)

```
app.MapGet("/", async ([FromServices]
IConnectionMultiplexer cache) =>
{
    var database = cache.GetDatabase();
    var messageKey = new RedisKey("Message");
    var lastUpdatedKey = new RedisKey("LastUpdated");
    if (!await database.KeyExistsAsync(messageKey))
        await database.StringSetAsync(messageKey, new
RedisValue("Hello, World!"));
    var message = (await
database.StringGetAsync(messageKey)).ToString();
    if (await database.KeyExistsAsync(lastUpdatedKey))
        message += " / " + (await
database.StringGetAsync(lastUpdatedKey)).ToString();
    return Results.Ok(message);
});
app.Run();
```

Minimal API에서는 Redis
멀티플렉서처럼 필요한 서비스를
먼저 등록해 두면, 컨트롤러
없이도 함수 매개변수로 간단히
주입 받아 사용할 수 있습니다.

만약 매개변수가 많아
복잡해진다면, 별도의 서비스
클래스로 캡슐화하거나,
IServiceProvider를 직접
사용해서 대체할 수도 있습니다.

파일 기반 마이크로 서비스 (feat. Aspire, Podman)

Aspire 샘플 (Worker.cs)

```
#!/usr/bin/env dotnet
#:sdk Microsoft.NET.Sdk.Worker
#:property PublishAot=false
#:package Microsoft.Extensions.Hosting@10.*
#:package Aspire.StackExchange.Redis@13.0.0

using StackExchange.Redis;

var builder = Host.CreateApplicationBuilder(args);
builder.Services.AddHostedService<Worker>();
builder.AddRedisClient("cache");

using var app = builder.Build();
app.Run();
```

Worker 프로젝트가 별도의 SDK를 사용하는 이유는, 일반 콘솔 앱과 달리 장시간 실행되는 백그라운드 프로세스 특성을 고려해 런타임 설정과 리소스 관리가 최적화되어 있기 때문입니다.

가비지 컬렉터 동작, 스레딩, 종료 신호 처리 등 장기 서비스 실행에 필요한 요소들이 템플릿 단계에서 미리 조정되어, 안정적으로 오랫동안 동작할 수 있도록 설계되어 있습니다.

파일 기반 마이크로 서비스 (feat. Aspire, Podman)

Aspire 샘플 (Worker.cs)

```
public sealed class Worker(
    ILogger<Worker> logger, IConnectionMultiplexer cache
) : BackgroundService
{
    protected override async Task
    ExecuteAsync(CancellationToken stoppingToken)
    {
        var key = new RedisKey("LastUpdated");
        var database = cache.GetDatabase();
        while (true)
        {
            var current = DateTimeOffset.Now;
            await database.StringSetAsync(key, "Last updated:
" + current.ToString());
            logger.LogInformation("Worker running at: {Time}",
current);
            await Task.Delay(TimeSpan.FromSeconds(1d),
stoppingToken);
        }
    }
}
```

BackgroundService 를 상속받아 ExecuteAsync 메서드를 구현하면, 콘솔 프로그램을 작성하듯 Worker 로직을 간단히 구성할 수 있습니다.

필요하다면 여러 개의 Worker 를 등록해 여러 종류의 백그라운드 작업을 동시에 실행하도록 구성하는 것도 가능합니다.

Demo: Aspire 13 + Podman 실행해보기

**파일 1 개로 만드는
크로스플랫폼 데스크톱 앱
(feat. Avalonia)**

파일 1 개로 만드는 크로스플랫폼 데스크톱 앱 (feat. Avalonia)

크로스플랫폼 UI 는 가까이 있습니다.

아주 실험적인 것부터 시작하여 , 지금은 안정적인 기술 체계를 갖춘 전설적인 UI 프레임워크가 하나 있습니다 . 바로 Avalonia 죠 !

Avalonia 는 XAML 의 Variant 인 AXAML 을 통해 WPF 개발자가 윈도우는 물론 리눅스 , 맥 OS 까지 한 번에 애플리케이션을 개발하고 배포할 수 있게 지원합니다 .

내부적으로는 Google 의 Skia 런타임을 사용하고 있고 , 요즈음은 Flutter 가 최근 채택하기 시작한 Impeller 도 도입을 앞두고 있습니다 .

파일 1 개로 만드는 크로스플랫폼 데스크톱 앱 (feat. Avalonia)

계산기 샘플

```
#!/usr/bin/env dotnet
#:property PublishAot=false
#:property OutputType=WinExe

#:package Microsoft.Extensions.Hosting@10.0.0
#:package Avalonia@11.3.8
#:package Avalonia.Desktop@11.3.8
#:package Avalonia.Themes.Simple@11.3.8
#:package Avalonia.Fonts.Inter@11.3.8
#:package CommunityToolkit.Mvvm@8.4.0

using Avalonia;
using Avalonia.Controls;
...
if (OperatingSystem.IsWindows())
{

Thread.CurrentThread?.SetApartmentState(ApartmentState.Unknown
);

Thread.CurrentThread?.SetApartmentState(ApartmentState.STA);
}
```

맥과 리눅스에서는 이 파일을
곧바로 GUI 애플리케이션으로
띄워볼 수 있어서 Shebang
라인을 사용합니다.

Avalonia Runtime 도 사용하고,
Generic Host 도 사용하면서
Modern 애플리케이션
아키텍처를 그대로 씁니다.

그리고 소스 제네레이터 지원을
이용해서 MVVM 도 구현할
것입니다.

파일 1 개로 만드는 크로스플랫폼 데스크톱 앱 (feat. Avalonia)

계산기 샘플

```
var builder = Host.CreateEmptyApplicationBuilder(new  
HostApplicationBuilderSettings() { Args = args, });  
builder.Services.AddSingleton<App>();  
builder.Services.AddSingleton<MainWindow>();  
builder.Services.AddSingleton<MainViewModel>();  
builder.Services.AddSingleton<CalculatorService>();
```

```
using var host = builder.Build();  
host.Start();
```

```
AppBuilder  
    .Configure(host.Services.GetRequiredService<App>) /*  
Delegate Pass */  
    .UsePlatformDetect()  
    .WithInterFont()  
    .LogToTrace()  
    .StartWithClassicDesktopLifetime(args, _ =>  
host.StopAsync().GetAwaiter().GetResult());
```

App, MainWindow,
MainViewModel,
CalculatorService 클래스를
싱글톤으로 DI 컨테이너에
추가합니다.

그리고 DI 컨테이너의 생명
주기를 활성화한 후, Avalonia 를
초기화합니다. Avalonia
런타임이 종료됨과 동시에 DI
컨테이너도 파기합니다.

파일 1 개로 만드는 크로스플랫폼 데스크톱 앱 (feat. Avalonia)

계산기 샘플

```
public partial class MainViewModel(CalculatorService
service) : ObservableObject
{
    [ObservableProperty]
    private string _display = "0";
    ...
    [RelayCommand]
    private void Calculate()
    {
        if (string.IsNullOrEmpty(Operation) ||
string.IsNullOrEmpty(FirstNumber))
            return;
        var result = service.Calculate(FirstNumber, Display,
Operation);
        Display = result;
        Operation = string.Empty;
        FirstNumber = string.Empty;
        IsNewNumber = true;
    }
    ...
}
```

뷰 모델은 `partial` 클래스로 선언하고, 이제껏 해왔던 것처럼 MVVM 클래스로서 선언합니다.

소스 제네레이터 지원도 정상적으로 포함되므로 알고 있던 코드 그대로 작성해도 빌드 오류가 발생하지 않습니다.

뿐만 아니라 C# Extension
에서도 이런 설정이 모두
인식됩니다.

파일 1 개로 만드는 크로스플랫폼 데스크톱 앱 (feat. Avalonia)

계산기 샘플

```
public class App : Application
{
    private readonly IServiceProvider _services;

    public App(IServiceProvider services)
    {
        _services = services;
        Styles.Add(new SimpleTheme());
    }

    public override void OnFrameworkInitializationCompleted()
    {
        if (ApplicationLifetime is
            IClassicDesktopStyleApplicationLifetime desktop)
            desktop.MainWindow =
                (MainWindow)_services.GetService(typeof(MainWindow))!;

        base.OnFrameworkInitializationCompleted();
    }
}
```

애플리케이션 클래스에서는
IServiceProvider 인스턴스를
넘겨받아 MainWindow 를 필요한
시점에 받아올 수 있게 관리합니다.

OnFrameworkInitialization
Completed 메서드가 호출되는
시점에 맞춰서 정확히
MainWindow 를 받아오도록
구성했습니다.

파일 1 개로 만드는 크로스플랫폼 데스크톱 앱 (feat. Avalonia)

계산기 샘플

```
public class MainWindow : Window
{
    public MainWindow(MainViewModel vm)
    {
        DataContext = vm;
        Title = "계산기";
        Width = 320;
        Height = 420;
        MinWidth = 280;
        MinHeight = 380;

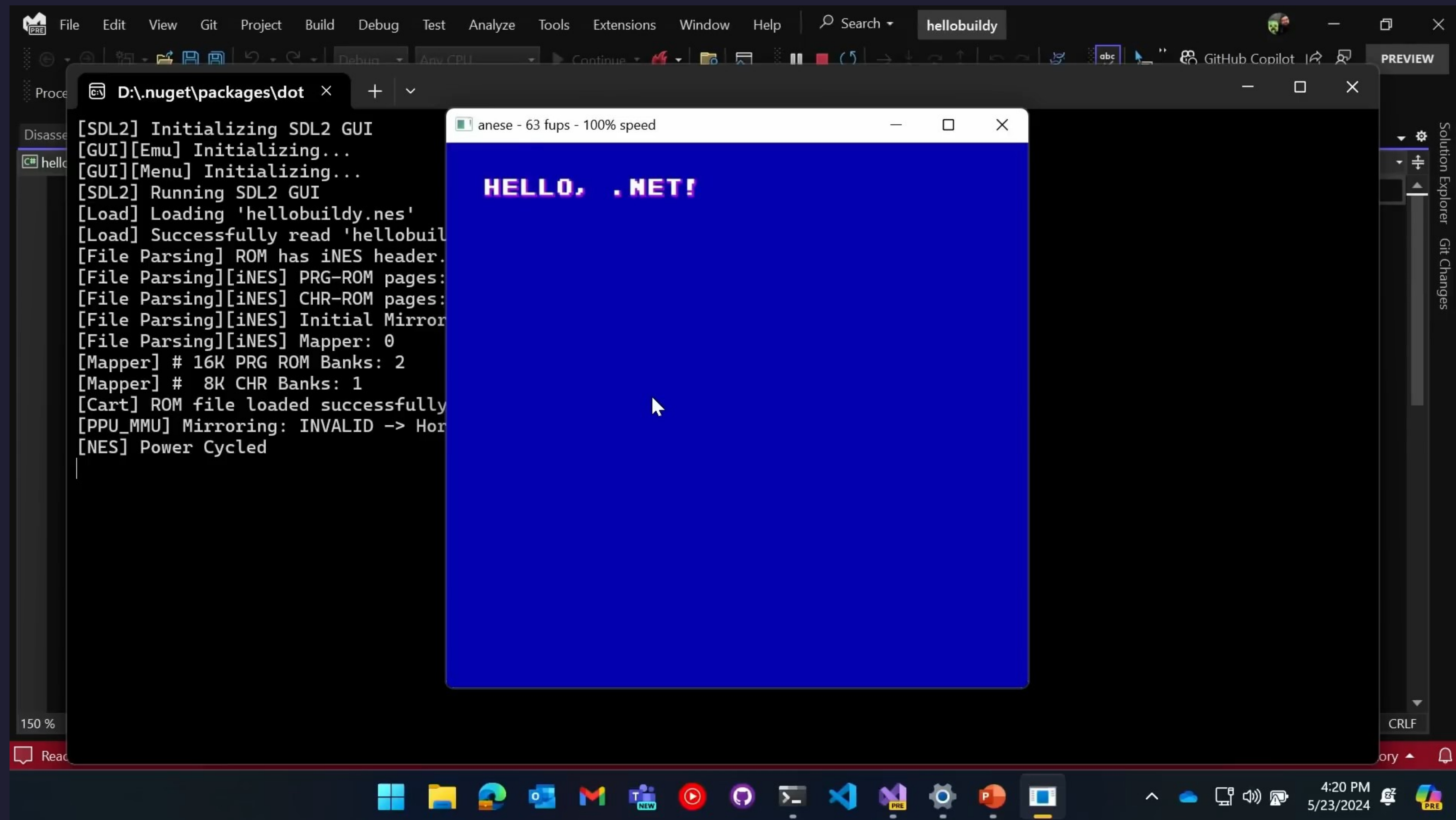
        var root = new Grid
        {
            Margin = new Thickness(10),
            RowDefinitions = new
RowDefinitions("Auto,*,*,*,*"),
            ColumnDefinitions = new
ColumnDefinitions("*,*,*,*"),
        };
        ...
    }
}
```

뷰 모델을 생성자 주입을 통해서 받은 후에는, 생성자 수준에서 곧바로 Avalonia 컨트롤들을 초기화하고 바인딩을 개시합니다.

아쉽게도 Avalonia 의 현재 버전에서는 파일 기반 앱에서 정적으로 편리하게 MVVM 바인딩을 지정할 방법을 찾지는 못해서 기존처럼 리플렉션 기반 바인딩을 사용하고 AOT 컴파일은 꺼 두었습니다.

Demo: Avalonia 계산기
앱 (맥에서 실행해보기)

C# 으로 패미콤 게임도
컴파일하는데 새삼스럽죠 ?



Microsoft Build 2024

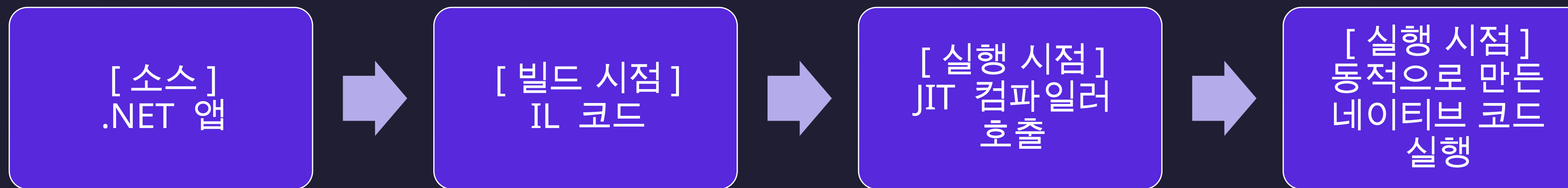
Running .NET on the NES | BRK252



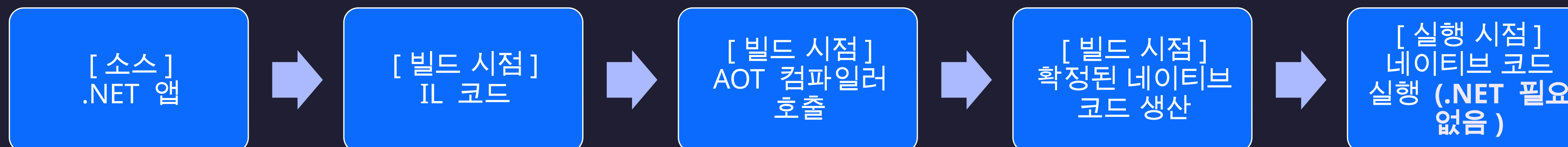
C# 으로 패미콤 게임도 컴파일하는데 새삼스럽죠 ?

Native AOT 란 ?

전통적인 .NET 런타임 실행 방식



Native AOT (Ahead-of-Time) 컴파일 방식



AOT 컴파일러가
만드는 코드를 실행할
때 .NET 은 전혀
필요하지 않습니다 .
그러나 이는...

C# 으로 패미콤 게임도 컴파일하는데 새삼스럽죠 ?

Native AOT 의 좋지만 불편한 점

좋은 점 : .NET 이 필요하지 않습니다 .

- 단순히 .NET 런타임을 압축 패키지로 묶어서 배포하는 게 아닙니다 .
- 필요 없는 코드는 전부 잘라버립니다 . (IL Trimming)
- 전부 특정 CPU 기계어 코드로 컴파일합니다 .
- 가비지 컬렉터는 기계어 코드 안에 같이 넣어줍니다 .
- 즉 , C, C++, Go, Rust 로 컴파일한 것과 비슷한 결과물이 나옵니다 .

불편한 점 : .NET 이 없습니다 .

- C# 으로 작성했을 뿐 , 더 이상 .NET 애플리케이션이 아니게 됩니다 .
- Reflection, IL Emit 은 거의 쓰지 못하거나 아예 못씁니다 .
- 다른 .NET 애플리케이션에서는 네이티브 애플리케이션처럼 다루어야 합니다 .
- 디버깅할 때 심벌 파일의 짝을 잘 맞춰서 배포하셔야 합니다 .

C# 으로 패미콤 게임도 컴파일하는데 새삼스럽죠 ?

애매한 것을 정해드립니다 .

닷넷은 JIT 때문에 못쓰다고 생각하시나요 ? AOT 컴파일러가 닷넷의 답입니다 .

Unity 처럼 IL 을 네이티브 코드로 바꾸기 원하시나요 ? C/C++ 코드를 만드는 과정이 들어있지는 않지만 , AOT 컴파일러가 닷넷의 답입니다 .

C, C++, Go, Python, Java, Swift 등 여러 프로그래밍 언어와 호환되는 DLL, SO, DYLIB 혹은 링커용 정적 라이브러리 (LIB, A) 를 만드셔야 하나요 ? AOT 컴파일러가 닷넷의 답입니다 .

그러나 위에서 해당되는 것이 없다면 , 굳이 AOT 컴파일러 때문에 고통받지 마시고 편하게 닷넷 쓰세요 !

C# 으로 패미콤 게임도 컴파일하는데 새삼스럽죠 ?

C# 으로 네이티브 함수 만들기

```
#:property OutputType=Library
#:property PublishAot=True
#:property RuntimeIdentifier=osx-arm64

using System.Runtime.InteropServices;

internal static class NativeEntryPoints
{
    [UnmanagedCallersOnly(EntryPoint = "mymethod")]
    public static void MyMethod(nint ptrText)
    {
        if (ptrText == default)
            return;

        string? text = Marshal.PtrToStringUni(ptrText);
        Console.WriteLine($"{DateTime.Now} {text}");
    }
}
```

OutputType 속성은 Library 로 ,
대상 OS 와 시스템을
RuntimeIdentifier 로 미리
정해줍니다 .

그리고 이런 목적으로 꺼내두는
메서드는 모두 static 으로
선언되어 있어야 하고 , 문자열
매개 변수를 주고 받을 때는
네이티브 포인터 연산을 사용해야
합니다 .

C# 으로 패미콤 게임도 컴파일하는데 새삼스럽죠 ?

Java 에서 C# DYLIB 쓰기

```
// .NET Native AOT 라이브러리 인터페이스
public interface NativeAotLib extends Library {
    // artifacts/05-native-aot/05-native-aot.dylib 로드
    // 절대 경로 사용
    String LIBRARY_PATH =
Paths.get(System.getProperty("user.dir"),
    "artifacts", "05-native-aot", "05-native-
aot.dylib").toString();
    NativeAotLib INSTANCE = Native.load(LIBRARY_PATH,
NativeAotLib.class);

    // void mymethod(void* ptrText)
    void mymethod(Pointer ptrText);
}
public static void callMyMethod(String text) {
    // UTF-16 LE 인코딩 + null terminator
    byte[] utf16Bytes =
text.getBytes(StandardCharsets.UTF_16LE);
    Memory memory = new Memory(utf16Bytes.length + 2);
    ...
    NativeAotLib.INSTANCE.mymethod(memory);
}
```

JNI, JNA 모두 사용 가능하지만 ,
편의를 위하여 JNA 사용을
추천합니다 .

C# 에서 네이티브 코드를 부를
때와 마찬가지로 네이티브
메모리에 대한 성형 / 구성 작업이
필요할 수 있습니다 .

C# 으로 패미콤 게임도 컴파일하는데 새삼스럽죠 ?

Python 에서 C# DYLIB 쓰기

```
# dylib 파일의 절대 경로 설정
script_dir = Path(__file__).parent
dylib_path = script_dir / "artifacts" / "05-native-aot" / "05-native-aot.dylib"

# dylib 로드
lib = ctypes.CDLL(str(dylib_path))

# mymethod 함수 설정
# void mymethod(nint ptrText)
mymethod = lib.mymethod
mymethod.argtypes = [ctypes.c_void_p]
mymethod.restype = None

# 유니코드 문자열을 UTF-16 으로 인코딩하여 포인터로 전달
def call_mymethod(text: str):
    # UTF-16 LE (Little Endian) 인코딩 + null terminator
    encoded = text.encode('utf-16-le') + b'\x00\x00'
    buffer = ctypes.create_string_buffer(encoded)
    mymethod(ctypes.cast(buffer, ctypes.c_void_p))
```

Python 의 ctypes.CDLL 확장을 사용하여 macOS 의 DYLIB 파일을 불러올 수 있습니다 .

C# 으로 패미콤 게임도 컴파일하는데 새삼스럽죠 ?

Node 에서 C# DYLIB 쓰기

```
const koffi = require('koffi');
const path = require('path');
const libraryPath = path.join(__dirname, 'artifacts', '05-native-aot', '05-native-aot.dylib');
const lib = koffi.load(libraryPath);
const mymethod = lib.func('mymethod', 'void', ['void*']);

function callMyMethod(text) {
  const buffer = Buffer.from(text, 'utf16le');
  const nullTerminator = Buffer.alloc(2);
  const combined = Buffer.concat([buffer, nullTerminator]);
  mymethod(combined);
}

callMyMethod('Hello from Node.js!');
callMyMethod('Native AOT 호출 테스트 ');
callMyMethod('안녕하세요 !');
```

Node 에서 koffi 를 사용하여
네이티브 동적 라이브러리를
불러오는 것도 가능합니다 .

C# 으로 패미콤 게임도 컴파일하는데 새삼스럽죠 ?

C 에서 C# 스택 라이브러리

쓰기

// Compile with:

```
// gcc -o 05-staticlib-c 05-staticlib-c.c artifacts/05-native-aot/05-native-aot.dylib -Wl,-rpath,artifacts/05-native-aot -liconv
```

// Run with:

```
// ./05-staticlib-c
```

// .NET Native AOT 에서 생성된 함수 선언

```
extern void mymethod(void*);
```

...

```
int main(int argc, const char** argv) {  
    call_mymethod("Hello from C!");  
    call_mymethod("Native AOT 호출 테스트");  
    call_mymethod("안녕하세요!");  
    printf("\nAll calls completed.\n");  
    return 0;  
}
```

믿기 어려우시겠지만 , Native AOT
가 단순히 DLL, SO, DYLIB 만
만드는게 아닙니다 .

LIB, A 파일도 만들어주고 ,
이것을 GCC 링커 , Microsoft VC+
+ 링커에 적절하게 옵션을 주면
C# 으로 만든 코드를 아예 다른
네이티브 프로그램의 일부로
그대로 편입시킬 수 있습니다 .

Demo: Native AOT 로 언어 / 런타임 경계를 넘나들기

AI 와 함께 춤을

AI 와 함께 춤을

AI 에이전트를 개발해야 하는데

AI· 에이전트 , 뜬구름 잡기 같은 이야기라 굉장히 막연합니다 .
그래서 AI 에이전트 비슷한 걸 만들어보려고 하면...

WebSocket? 상태 동기화 수동 처리? 도구 호출 구현? AI API 쓰는 방법?
AI 에이전트를 만드는 본질은 사라지고 세부 사항만 탐독하게 되서 힘듭니다 .

이런 문제를 해결하고 , AI 상호 작용의 표준 프로토콜을 정의한 것이 AG-UI 입니다 .

AI 와 함께 춤을

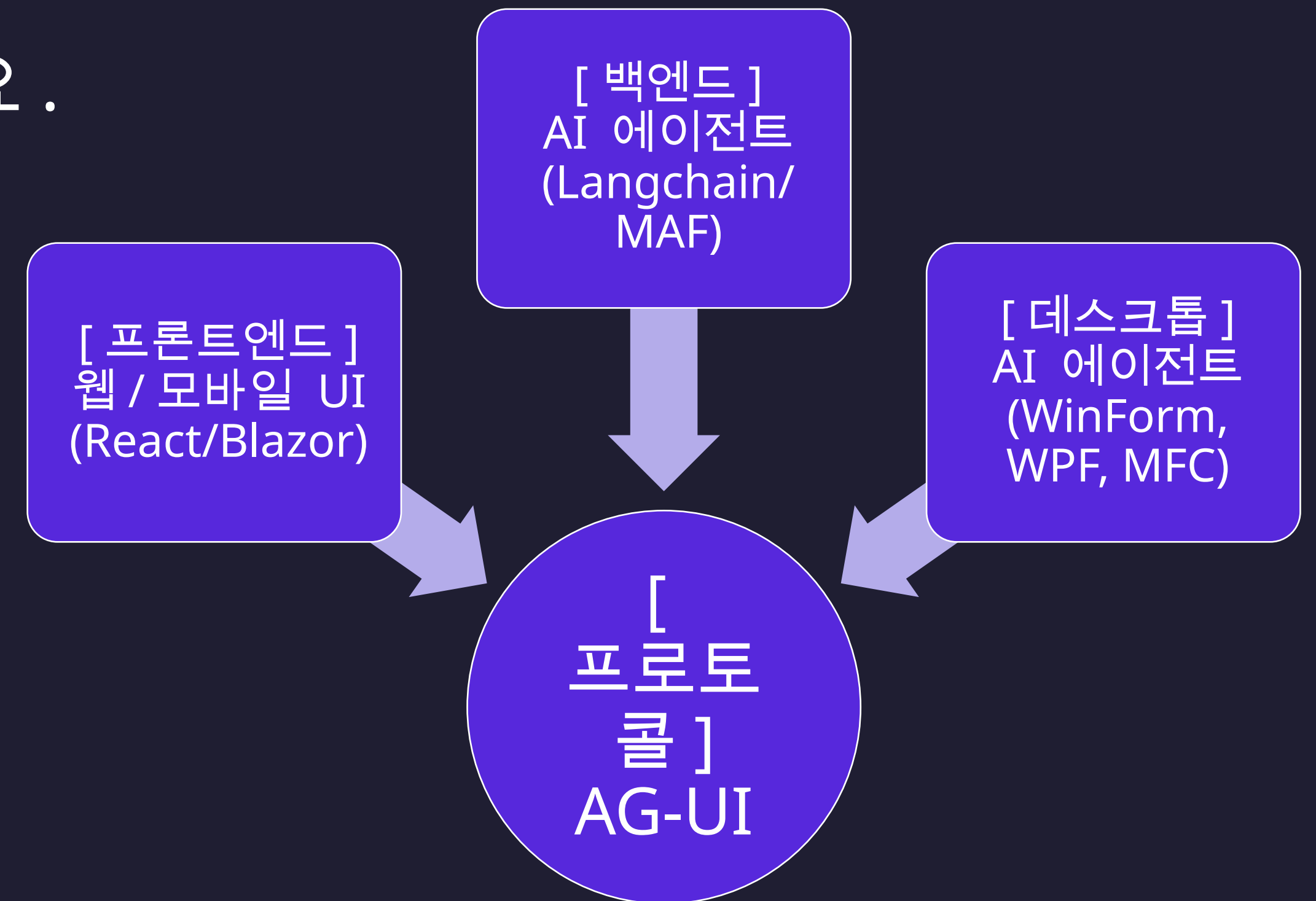
AG-UI 란 ?

AG-UI 는 AI 에이전트와 UI 를 연결하는 표준 프로토콜입니다 .

여러분 , 혹은 다른 사람이 어떤 언어 , 도구 , 프레임워크를 써서 에이전트를 만들었든 상관 없습니다 .

더 이상 SignalR/WebSocket 을 직접 짜지 마세요 .

AG-UI 가 표준 방식을 제공합니다 .



그런데 MCP 랑 뭐가 달라요 ?

AG-UI 는 모든 소프트웨어가 AI 에이전트와 통신하는 방법을 표준화한 것입니다 .
MCP 는 에이전트가 쓸 수 있는 내부 도구를 부를 수 있는 방법을 표준화한 것입니다 .

❶ 사용자가 질문 :

" 오늘 서울 날씨 어때 ? " ① [AG-UI] 질문을 에이전트에게 전달

❷ 에이전트가 도구 호출 :

" 날씨 API 를 호출해야겠다 " ① [MCP] 날씨 API 서버에 요청 ② [MCP] 응답 : " 맑음 , 25 도 "

❸ 에이전트가 응답 생성 :

" 서울은 맑고 25 도입니다 " ① [AG-UI] 실시간 스트리밍으로 UI 에 표시

즉 , 사용자 ↔ [AG-UI] ↔ 에이전트 ↔ [MCP] ↔ 도구 / 데이터 (RAG/DB) 관계입니다 .

AI 와 함께 춤을

ASP .NET Core 에 AG-UI 통합하기

```
var builder = WebApplication.CreateBuilder(args);
builder.Configuration.AddUserSecrets("06-agui-server");
builder.Services.AddSingleton<MathTools>();
builder.Services.AddChatClient(
    new OpenAIClient(
        new ApiKeyCredential(builder.Configuration["openrouter-
key"] ?? throw new Exception("openrouter-key is missing.")),
        new OpenAIClientOptions { Endpoint = new
Uri("https://openrouter.ai/api/v1", UriKind.Absolute), }
    ).GetChatClient("x-ai/grok-4.1-fast").AsIChatClient());

using var app = builder.Build();
var mathTool = app.Services.GetRequiredService<MathTools>();
var agent =
app.Services.GetRequiredService<IChatClient>().CreateAIAgent(
    name: "AGUIAssistant",
    instructions: "You are a helpful assistant.",
    tools: [
        AIFunctionFactory.Create(mathTool.Add, nameof(mathTool.Add),
"Add two numbers"),
    ]);
app.MapAGUI("/aguitest", agent);
app.Run();
```

Microsoft Agent Framework 의
일부로서 제공되는 AG-UI
구현체는 ASP .NET Core Kestrel
에 자연스럽게 통합됩니다 .

MapAGUI 확장 메서드로 Agent
클래스를 지정하면 곧바로 AG-UI
인터페이스가 공개되어
클라이언트에서 쓸 수 있게 됩니다 .

AI 와 함께 춤을

ASP .NET Core 에 AG-UI

통합하기

```
public class MathTools(ILogger<MathTools> logger)
{
    public int Add(int a, int b)
    {
        logger.LogInformation("Add tool called: {a}, {b}", a,
b);
        return a + b;
    }

    public int Sub(int a, int b)
    {
        logger.LogInformation("Subtool called: {a}, {b}", a,
b);
        return a - b;
    }
}
```

그리고 흔히 보는 C# 버전의 Agent Framework 코드처럼 작성하지 않고, 이와 같이 의존성 주입을 사용하는 서비스 클래스로도 만들 수 있습니다.

여기서 중요한 포인트!

Add, Sub 메서드에 인자가 어떻게 오든 의미만 맞으면 나머지는 LLM AI 가 Tool Calling 을 대행해줍니다.

AI 와 함께 춤을

클라이언트에서 접근하기

```
var serverUrl = "http://localhost:5000/aguitest";
Console.WriteLine($"Connecting to AG-UI server at:
{serverUrl}\n");
using var httpClient = new HttpClient()
{
    Timeout = TimeSpan.FromSeconds(60)
};
AGUIChatClient chatClient = new(httpClient, serverUrl);

AIAgent agent = chatClient.CreateAIAgent(
    name: "agui-client",
    description: "AG-UI Client Agent",
    tools: []);

AgentThread thread = agent.GetNewThread();
List<ChatMessage> messages =
[
    new(ChatRole.System, "You are a helpful assistant.")
];
```

AG-UI 방식으로 에이전트를 웹 서버에서 드러내면 SSE 통신 채널이 열립니다 .

이것을 클라이언트에서 “웹 주소 방식”으로 접근해서 ChatGPT 쓰듯 연동할 수 있게 됩니다 .

그리고 클라이언트도 자신이 쓸 수 있는 도구를 서버에서 했던 것처럼 정의할 수 있습니다 .

Demo: AG-UI Tool

Calling 데모 살펴보기

마치|며

마치며

파일 기반 앱의 프로젝트 확장

파일 기반 앱은 AI 코딩 어시스턴트와 잘 맞습니다.
파일 1 개로 모든 것을 끝낼 수 있으므로 바이트코딩에 유리합니다.

+

파일 기반 앱은 내부적으로 여전히 프로젝트 개념을 사용합니다.
당연히 csproj 로 승격하는 것이 아래 명령어 하나면 가능합니다.

```
$ dotnet project convert Program.cs
```

충분히 파일 기반 앱에서 개발이 끝났다면 이렇게 개발을 시작하세요.

마치며

더 쉽게 FBA 시작하기

파일 기반 앱을 더 빠르고 쉽게 개발하고 싶으시다면?

```
$ dotnet new install FbaTemplates
```

FbaTemplates 만 설치하시면 손쉽게 FBA 코드를 만드실 수 있습니다.

github.com/dotnetdev-kr/dotnet-fba-templates



마치며

.NET 은 계속 성장하고 있습니다 .

오늘 보여드린 것은 .NET 의 극히 일부일 뿐입니다 .

.NET 은 더 이상 Microsoft 가 혼자 개발하는 기술이 아닙니다 .

**.NET Foundation 과 수많은 글로벌 오픈소스 기여자 · 기업들이
함께 만들어가는 전 세계가 함께 만드는 공개 · 오픈 기술입니다 .**

**남들과 다른 커리어 상의 차별점을 원하시나요 ?
새로운 기술을 배우고 싶으신가요 ?**

**그렇다면 지금이 C# 과 .NET 을 배울 때입니다 .
그리고 , 함께 만들 때입니다 .**

마치며

오늘 보여드린 코드 샘플은
이곳에서!

github.com/dotnetdev-kr/dotnet-fba-examples

위의 URL 또는 우측의 QR Code 를 스캔하시면
오늘 다룬 코드 샘플을 모두 살펴보실 수 있습니다!



마치며

.NET Universe 2026 Seoul

오는 2026년 3월 12일 삼성역
스페이스췌어에서

.NET Universe 2026 Seoul 이 열립니다 .
후원사 , 발표자 참여를 기다리고 있습니다 !

dotnetuniverse.net/2026



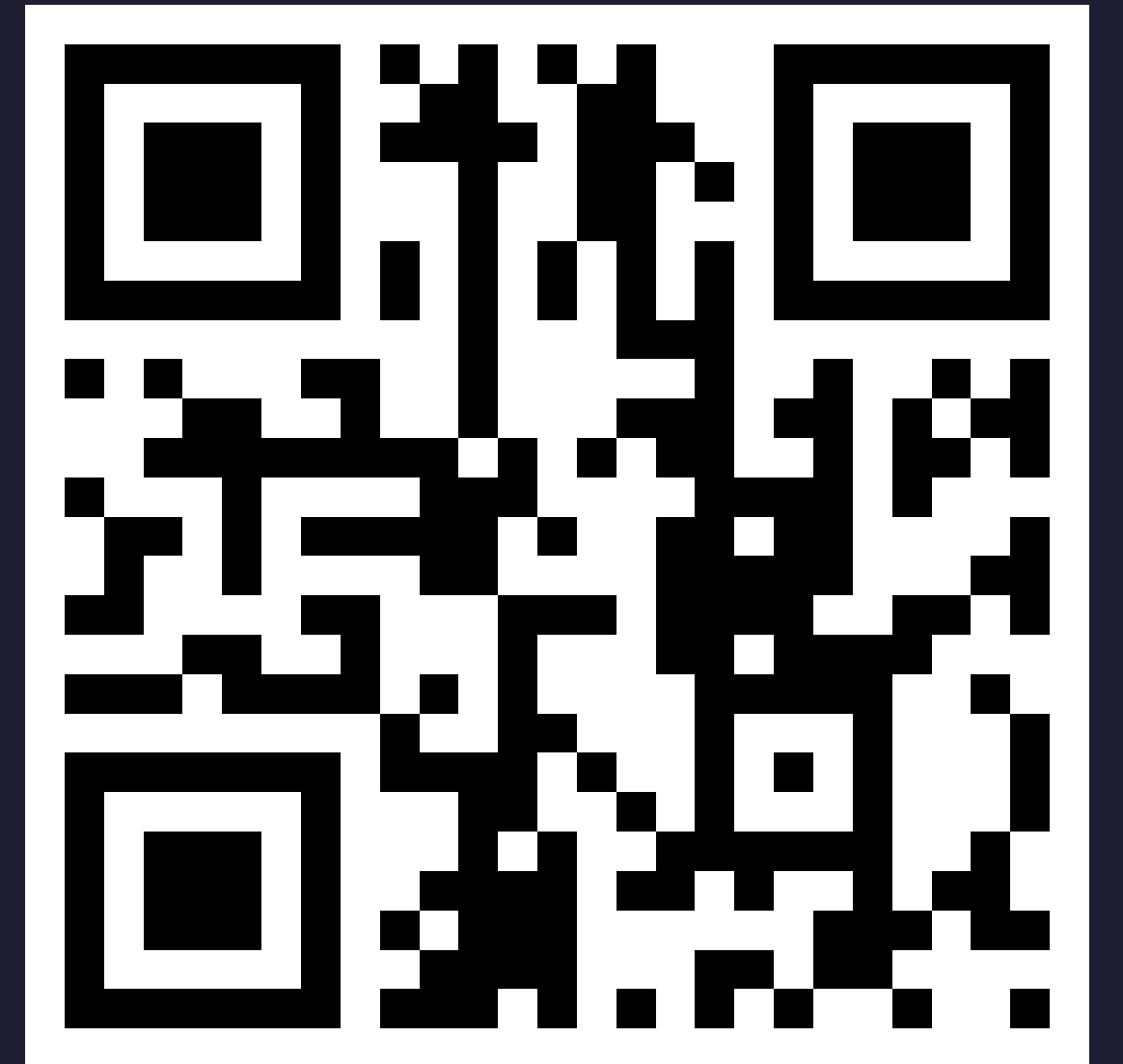
마치며

닷넷데브

2019 년부터 국내 .NET 개발자 생태계를 대표해
온 비영리 기술 단체 , 닷넷데브

우리는 .NET 개발자가 차별 받지 않는 생태계를
만듭니다 .
함께 해주세요 ! 😊

forum.dotnetdev.kr



고맙습니다!

[linkedin.com/in/
rkttu](https://linkedin.com/in/rkttu)

